

Importing Required Libraries

```
# Data Handling
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

Uploading Dataset

```
from google.colab import files
uploaded = files.upload()

<IPython.core.display.HTML object>

Saving WA_Fn-UseC_-HR-Employee-Attrition.csv to WA_Fn-UseC_-HR-
Employee-Attrition.csv

df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")
```

Data Understanding

```
df.head()

{"type": "dataframe", "variable_name": "df"}

df.shape

(1470, 35)

df.columns
```

```
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate',
      'Department',
      'DistanceFromHome', 'Education', 'EducationField',
      'EmployeeCount',
      'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender',
      'HourlyRate',
      'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',
      'MaritalStatus', 'MonthlyIncome', 'MonthlyRate',
      'NumCompaniesWorked',
      'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating',
      'RelationshipSatisfaction', 'StandardHours',
      'StockOptionLevel',
      'TotalWorkingYears', 'TrainingTimesLastYear',
      'WorkLifeBalance',
      'YearsAtCompany', 'YearsInCurrentRole',
      'YearsSinceLastPromotion',
      'YearsWithCurrManager'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1470 entries, 0 to 1469
```

```
Data columns (total 35 columns):
```

#	Column	Non-Null Count	Dtype
0	Age	1470 non-null	int64
1	Attrition	1470 non-null	object
2	BusinessTravel	1470 non-null	object
3	DailyRate	1470 non-null	int64
4	Department	1470 non-null	object
5	DistanceFromHome	1470 non-null	int64
6	Education	1470 non-null	int64
7	EducationField	1470 non-null	object
8	EmployeeCount	1470 non-null	int64
9	EmployeeNumber	1470 non-null	int64
10	EnvironmentSatisfaction	1470 non-null	int64
11	Gender	1470 non-null	object
12	HourlyRate	1470 non-null	int64
13	JobInvolvement	1470 non-null	int64
14	JobLevel	1470 non-null	int64
15	JobRole	1470 non-null	object
16	JobSatisfaction	1470 non-null	int64
17	MaritalStatus	1470 non-null	object
18	MonthlyIncome	1470 non-null	int64
19	MonthlyRate	1470 non-null	int64
20	NumCompaniesWorked	1470 non-null	int64
21	Over18	1470 non-null	object
22	OverTime	1470 non-null	object
23	PercentSalaryHike	1470 non-null	int64

24	PerformanceRating	1470	non-null	int64
25	RelationshipSatisfaction	1470	non-null	int64
26	StandardHours	1470	non-null	int64
27	StockOptionLevel	1470	non-null	int64
28	TotalWorkingYears	1470	non-null	int64
29	TrainingTimesLastYear	1470	non-null	int64
30	WorkLifeBalance	1470	non-null	int64
31	YearsAtCompany	1470	non-null	int64
32	YearsInCurrentRole	1470	non-null	int64
33	YearsSinceLastPromotion	1470	non-null	int64
34	YearsWithCurrManager	1470	non-null	int64

dtypes: int64(26), object(9)

memory usage: 402.1+ KB

df.describe()

```
{"type": "dataframe"}
```

df.isnull().sum()

Age	0
Attrition	0
BusinessTravel	0
DailyRate	0
Department	0
DistanceFromHome	0
Education	0
EducationField	0
EmployeeCount	0
EmployeeNumber	0
EnvironmentSatisfaction	0
Gender	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobRole	0
JobSatisfaction	0
MaritalStatus	0
MonthlyIncome	0
MonthlyRate	0
NumCompaniesWorked	0
Over18	0
OverTime	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StandardHours	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0

```

WorkLifeBalance      0
YearsAtCompany        0
YearsInCurrentRole    0
YearsSinceLastPromotion 0
YearsWithCurrManager  0
dtype: int64

df.fillna(method='ffill',inplace=True)
df.duplicated().sum()
np.int64(0)

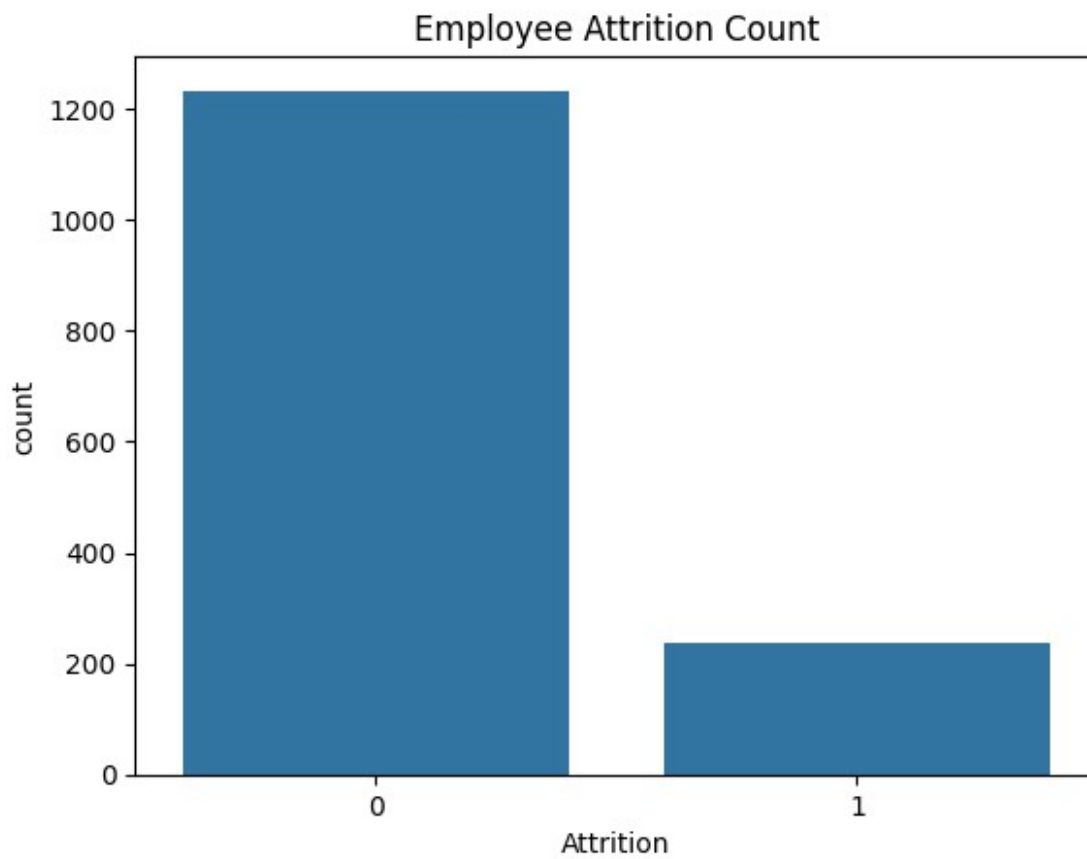
df.drop_duplicates(inplace=True)

le = LabelEncoder()

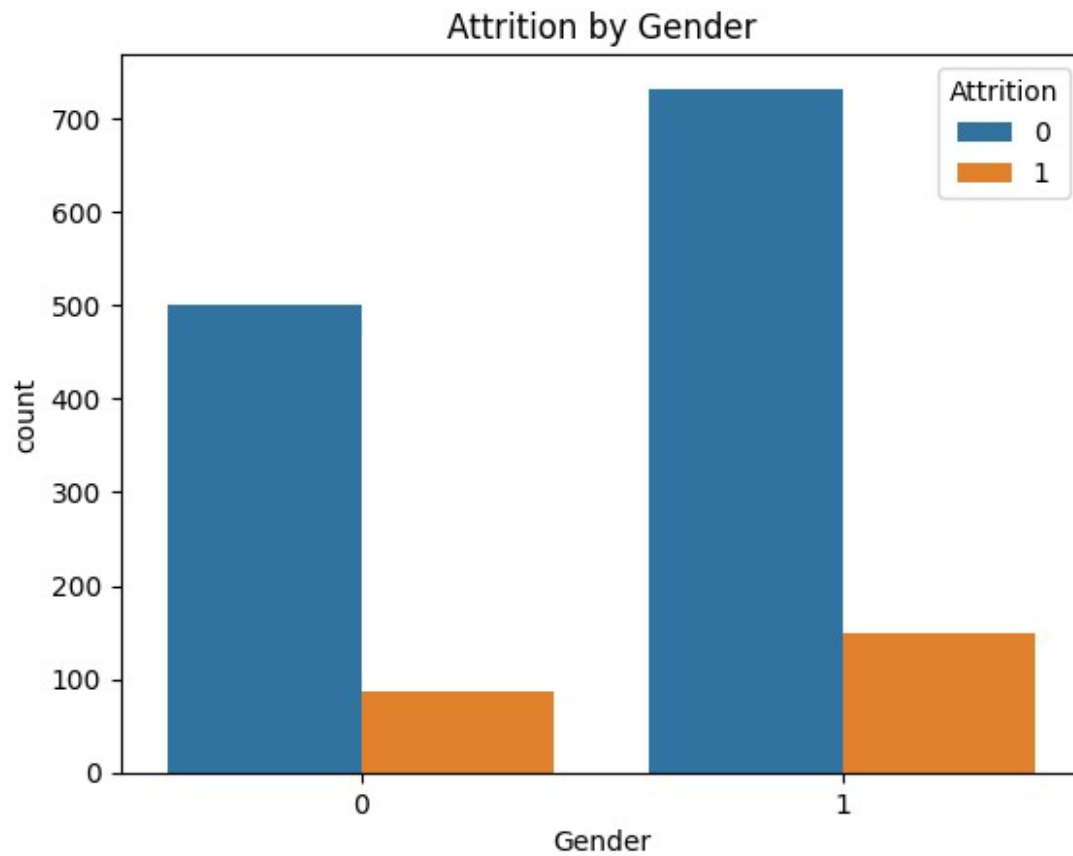
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

sns.countplot(x='Attrition', data=df)
plt.title("Employee Attrition Count")
plt.show()

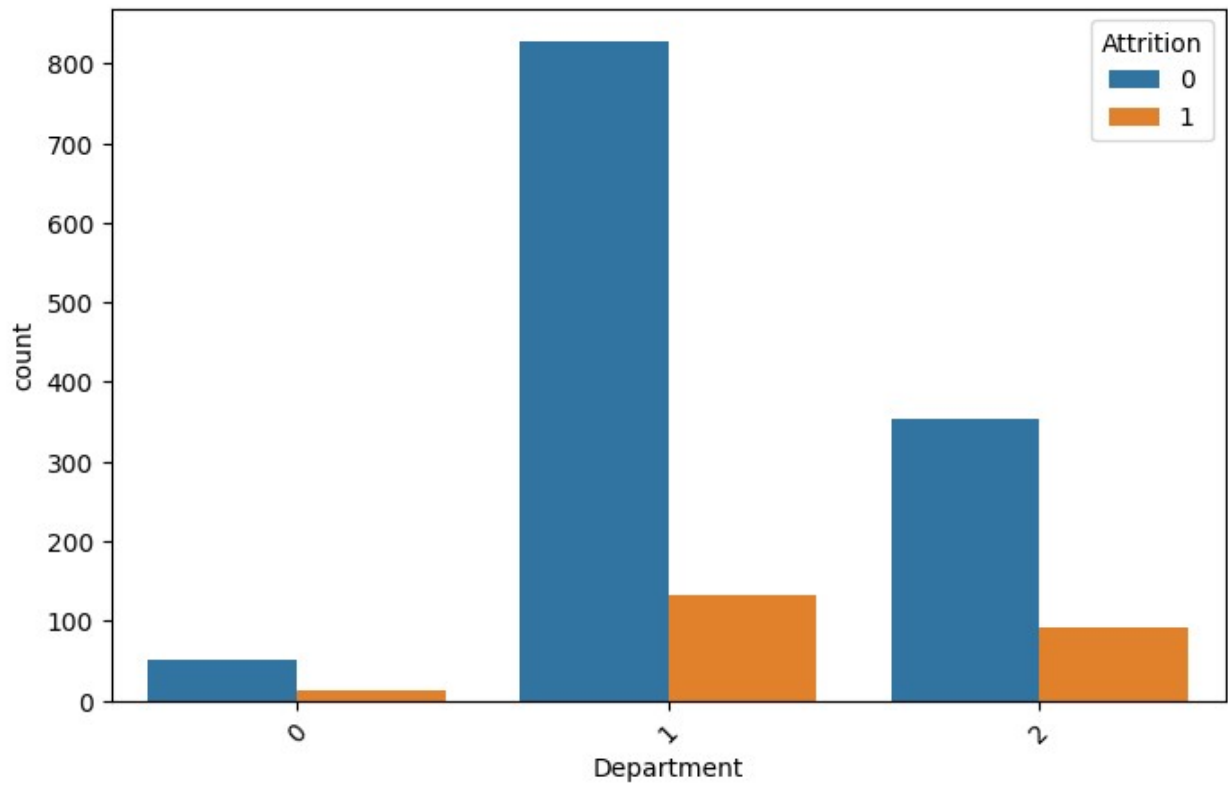
```



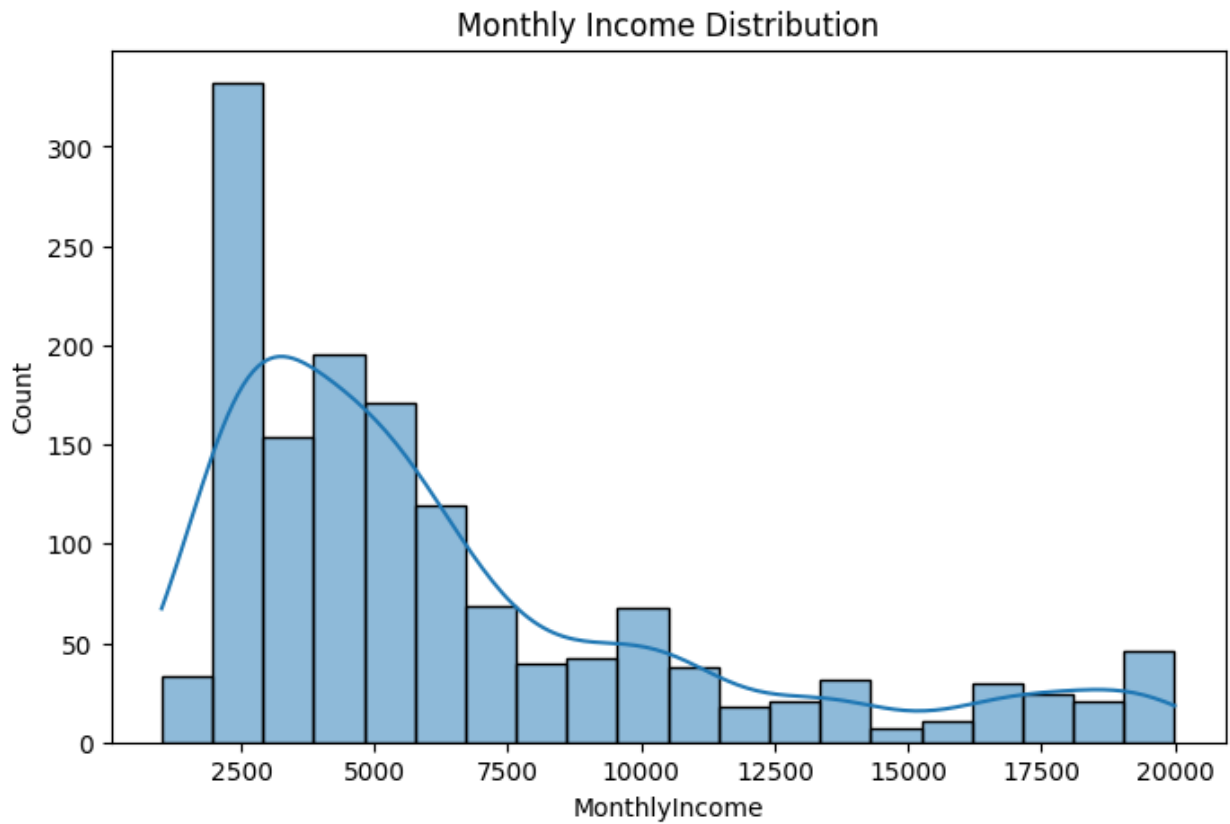
```
sns.countplot(x='Gender', hue='Attrition', data=df)
plt.title("Attrition by Gender")
plt.show()
```



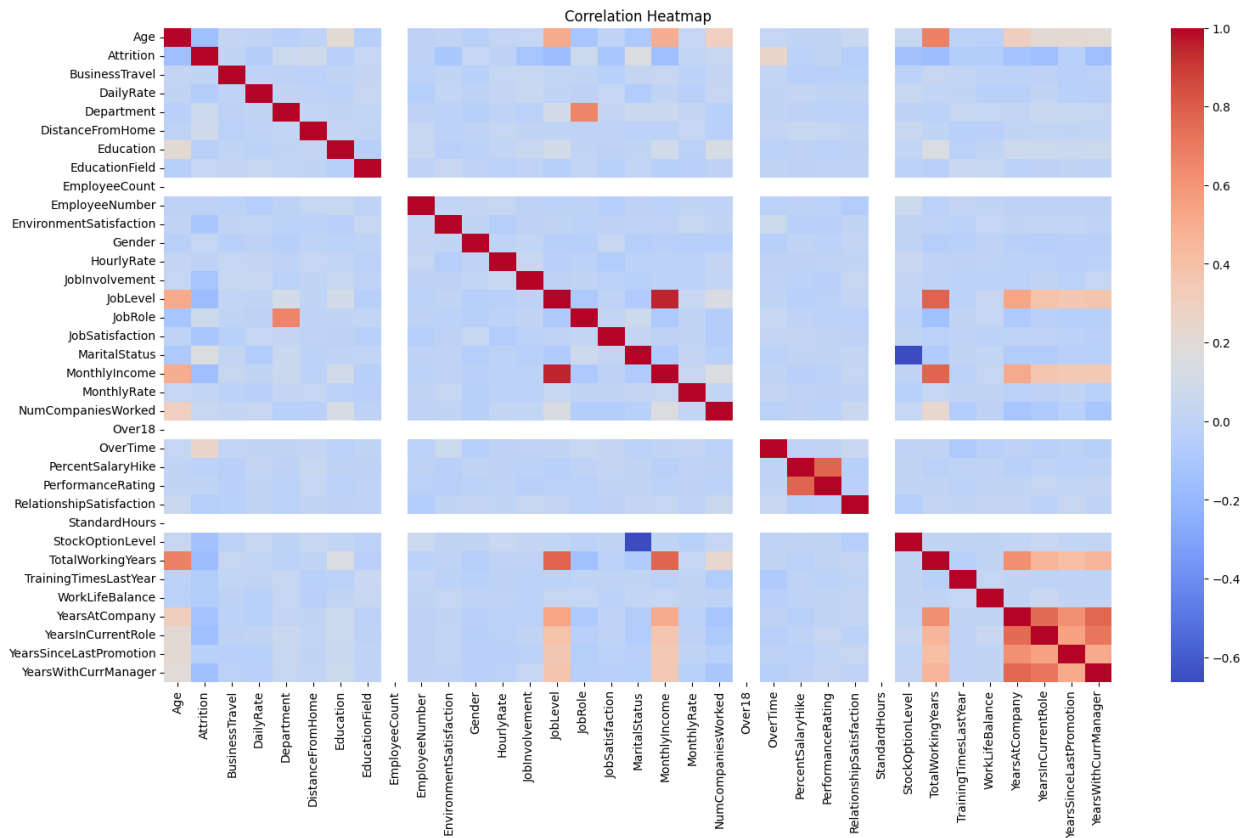
```
plt.figure(figsize=(8,5))
sns.countplot(x='Department', hue='Attrition', data=df)
plt.xticks(rotation=45)
plt.show()
```



```
plt.figure(figsize=(8,5))
sns.histplot(df['MonthlyIncome'], kde=True)
plt.title("Monthly Income Distribution")
plt.show()
```



```
plt.figure(figsize=(18,10))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



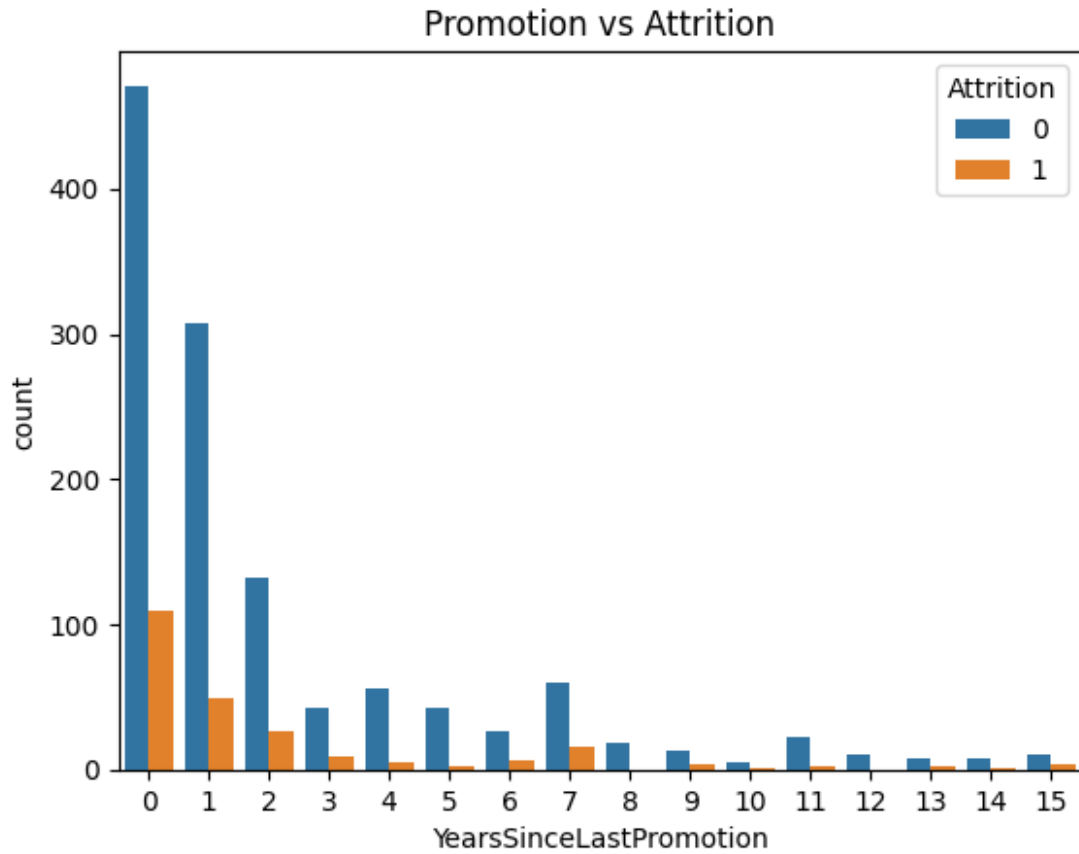
```
sns.countplot(x='WorkLifeBalance', hue='Attrition', data=df)

plt.title("Work Life Balance vs Attrition")

plt.show()
```




```
sns.countplot(x='YearsSinceLastPromotion', hue='Attrition', data=df)
plt.title("Promotion vs Attrition")
plt.show()
```



```
X = df.drop('Attrition', axis=1)
y = df['Attrition']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
model = RandomForestClassifier()
model.fit(X_train, y_train)
RandomForestClassifier()
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
Accuracy: 0.8741496598639455
```

```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.88	0.99	0.93	255
1	0.67	0.10	0.18	39
accuracy			0.87	294
macro avg	0.77	0.55	0.55	294
weighted avg	0.85	0.87	0.83	294

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6,4))
```

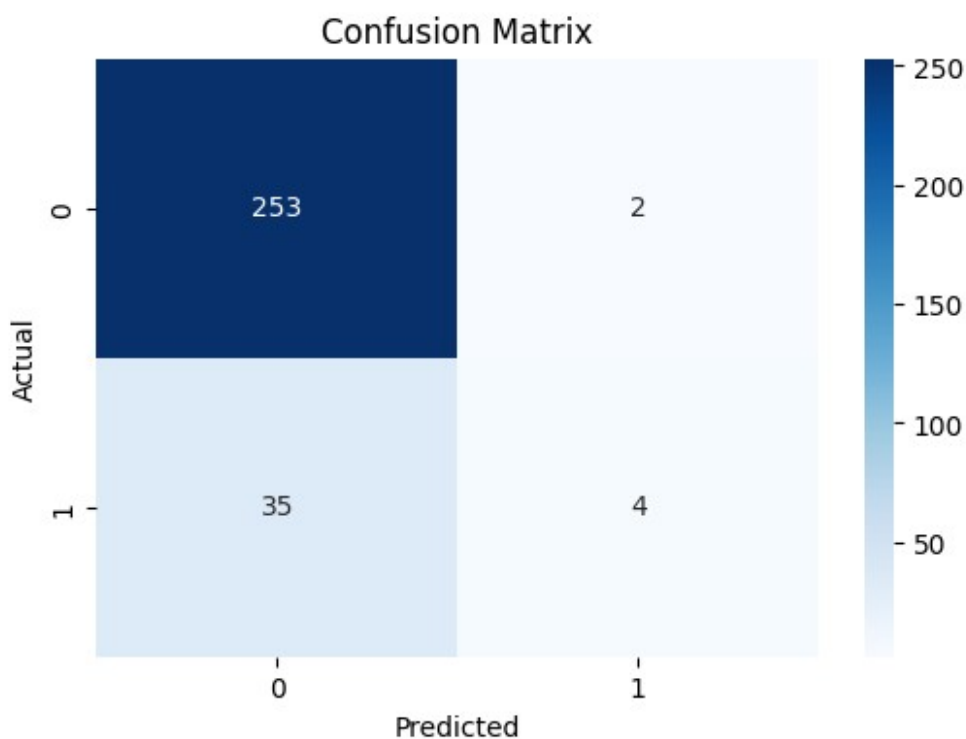
```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```



```
importance = pd.DataFrame({  
    'Feature': X.columns,
```

```

    'Importance': model.feature_importances_
})

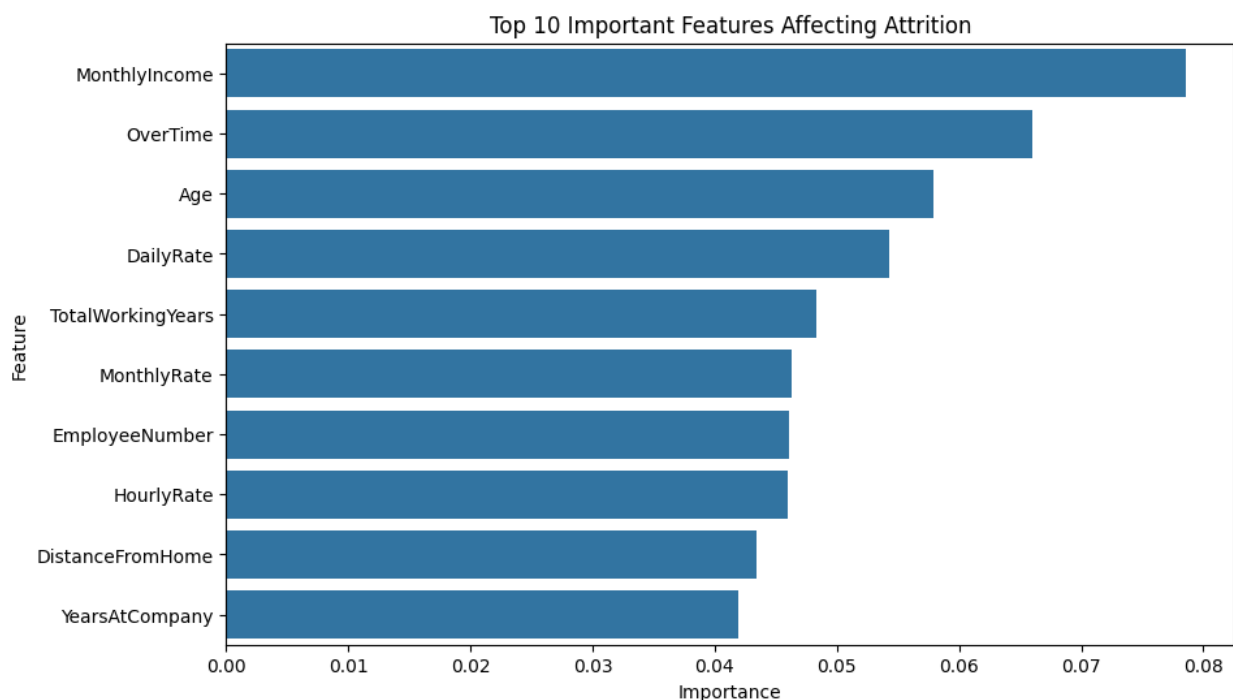
importance = importance.sort_values(
    by='Importance',
    ascending=False
)

plt.figure(figsize=(10,6))

sns.barplot(
    x='Importance',
    y='Feature',
    data=importance.head(10)
)

plt.title("Top 10 Important Features Affecting Attrition")
plt.show()

```



Key Business Insights

- 1-Employees working overtime are more likely to leave the company.
- 2- Lower job satisfaction increases employee attrition.
- 3-Employees with lower monthly income show higher resignation rates.

4-Lack of promotions contributes to attrition.

5-Work-life balance strongly impacts employee retention.

Business Recommendation

1-Improve work-life balance policies

2-Reduce employee overtime

3-Enhance employee engagement programs

4-Offer better compensation and incentives

5-Provide career growth opportunities

6-Focus on retention strategies in high attrition departments

Conclusion

The HR Attrition Prediction project successfully identified major factors influencing employee attrition. Overtime, low salary, poor work-life balance, and low job satisfaction were key reasons behind employee exits. The predictive model can help HR departments improve retention strategies and reduce employee turnover.